நிரல்15.1 செயற்கூறு பணிமிகுப்பினை விளக்கும் எளிய C++ நிரலை கீழே காணலாம்.

```
#include <iostream>
using namespace std;
void print(int i)
 {cout<< " It is integer " << i <<endl;}
void print(double  f)
{  cout<< " It is float " << f <<endl;}
void print(string c)
{  cout<< " It is string " << c <<endl;}
int main() {
        print(10);
        print(10.10);
        print("Ten");
        return 0;
}
```

வெளியீடு

```
        It is integer 10
        It is float 10.1
        It is string Ten
```

குறிப்பு:-

செயற்கூறு பணிமிகுப்பு, பல்லுருவாக்கத்தை மட்டுமே நடைமுறைப்படுத்தாமல் ஓர் நிரலில் ஒப்பீடுகளின் எண்ணிக்கையைக் குறைத்து, நிரல் வேகமாக செயல்பட உதவுகிறது. நிரலர், அதிக செயற்கூற்றின் பெயர்களை நினைவில் வைத்துக் கொள்வதை தவிர்க்க வழி செய்கிறது.

15.2.2 செயற்கூறு பணிமிகுப்பிற்கான விதிமுறைகள் (Rules for function overloading)

1. பணிமிகுத்த செயற்கூறுகள் முறையான அளபுருக்களின் எண்ணிக்கையிலோ, அல்லது அவற்றின் தரவு இனங்களிலோ வேறுபட்டிருக்க வேண்டும்.
2. பணிமிகுத்த செயற்கூறுகள் திருப்பியனுப்பும் தரவினம் ஒன்றாக இருக்க வேண்டும் என்ற தேவையில்லை.
3. பணிமிகுத்த செயற்கூறுகளின் தானமைவு செயலுருபுகளை அளபுருக்களின் பட்டியலில் ஒரு பகுதியாக C++ நிரல் பெயர்ப்பி கருதிக் கொள்ளாது.

நிரல் 15.2 செயற்கூறு பணிமிகுப்பினை விளக்கும் C++ நிரலை கீழே காணலாம்

```
#include <iostream>
using namespace std;
long add(long, long);
long add(long,long,long);
float add(float, float);
int main()
{
 long a, b, c,d;
 float e, f, g;
 cout << "Enter three integers\n";
cin >> a >> b>>c;
d=add(a,b,c);   //number of arguments different  but  same data type
cout << "Sum of 3 integers: " << d << endl;
cout << "Sum of 3 integers: " << d << endl;
cout << "Enter two integers\n";
cin >> a >> b;
c = add(a, b);   //two arguments data type same with above function call and
different with below function call
cout << "Sum of 2 integers: " << c << endl;
cout << "Enter two floating point numbers\n";
```


Chapter 15.indd 249 20-12-2022 15:34:26